

AKSHAR AI

SUMMER INTERNSHIP REPORT

WEEK 6 | 27 JUNE 2026 – 03 JULY 2026

Project Title: Pneumonia Detection System & Human Behavior Tracking System

INTERN DETAILS

Name	Yashas Raj R
Enrollment No.	24162121033
Branch	BDA (B.Tech – Computer Science and Engineering)
Institute	ICT Ganpat University, Mehsana, Gujarat
Organisation	Akshar AI Ltd
Report Period	29 June 2026 – 04 July 2026 (Week 6)
Email	24162121033@gnu.ac.in
Project Title	Pneumonia Detection System / Hand & Gesture Recognition

1. Introduction

Week 6 of the Akshar AI Summer Internship represented the zenith of the project lifecycle. Following five weeks of intense theoretical research, architectural design, dataset curation, and massive-scale empirical model training, this week was entirely dedicated to the complex processes of system integration, web deployment, and final project delivery. The culmination of both the Pneumonia Detection System and the foundational elements of the Human Behavior Tracking System occurred within this high-intensity phase.

The primary directive for the Pneumonia Detection System was to transition the highly accurate, 1000-image trained MobileNetV2 model from a static, local Python environment into a dynamic, user-facing web application. This necessitated the architecture and development of a robust backend API utilizing the Flask framework to handle image ingestion and tensor inference. Concurrently, a modern, responsive frontend interface had to be developed using HTML, CSS, and JavaScript to provide medical professionals with an intuitive platform for uploading chest X-rays and instantly receiving diagnostic predictions.

Simultaneously, the Hand & Gesture Recognition Module, which saw its foundational real-time tracking pipeline established in Week 5, required significant algorithmic expansion. The objective was to evolve the system from merely drawing lines on a screen to semantically interpreting the kinematic data. This involved writing complex mathematical heuristics to calculate Euclidean distances between the 3D landmarks, thereby enabling the reliable classification of discrete gestures (e.g., Open Palm, Closed Fist, specific finger counts).

This extensive final report details the comprehensive full-stack development, the algorithmic leaps in gesture recognition, and the rigorous system testing conducted throughout Week 6. It chronicles the day-by-day progression from backend routing to frontend styling, culminating in the final deployment and presentation of the cohesive software solutions. It serves as the definitive technical record of the project's completion, highlighting the successful translation of abstract machine learning models into tangible, impactful software.

2. Weekly Objectives

The objectives for Week 6 were uniquely focused on Software Engineering (SWE) and Full-Stack Development, shifting away from pure data science to prioritize deployment, API architecture, and algorithmic heuristic design.

The primary objectives for Week 6 were defined as follows:

1. **Flask Backend Architecture:** Design and implement a robust RESTful API using the Python Flask framework. The backend must be capable of receiving multi-part form data (images),

preprocessing the image to match the MobileNetV2 input tensor requirements, executing the inference, and returning a JSON response.

2. **Frontend UI/UX Development:** Develop a clean, responsive, and intuitive web interface using HTML5, vanilla CSS3, and JavaScript. The interface must allow users to upload chest X-ray images, provide visual loading feedback during inference, and clearly display the final diagnostic prediction (Normal vs. Pneumonia) along with confidence metrics.
3. **Full-Stack Integration:** Seamlessly connect the frontend client to the Flask backend API. Ensure robust error handling for unsupported file types, network timeouts, and malformed image tensors.
4. **Gesture Recognition Heuristics:** Advance the Hand Tracking module by implementing mathematical heuristic algorithms. Calculate the Euclidean distances between the MediaPipe 3D landmarks (specifically the fingertips and palm base) to classify at least three distinct gestures in real-time.
5. **System Optimization and Refactoring:** Conduct a comprehensive code review of both projects. Refactor redundant code, optimize imports, and ensure the deployment environment is clean and efficient.
6. **Final Project Documentation:** Consolidate the entire 6-week journey. Finalize all README files, architectural diagrams, and prepare the codebase for official handover to the Akshar AI engineering team.
7. **Final Presentation:** Prepare and deliver a comprehensive presentation demonstrating the fully functional web application and the real-time gesture recognition system to the mentorship team.

Day-wise Objective Summary

Day	Date	Primary Objective	Key Deliverable
1	27 Jun 2026 (Sat)	Exporting the optimal model & Architecting the Flask backend	Functional API endpoint /predict handling image tensors
2	29 Jun 2026 (Mon)	Designing the Frontend UI (HTML/CSS) for the medical web app	Responsive, styled user interface for image uploads
3	30 Jul 2026 (Tue)	Full-Stack Integration (JavaScript Fetch API to Flask backend)	Fully operational end-to-end Pneumonia Detection Web App
4	01 Jul 2026 (Wed)	Developing Euclidean distance heuristics for Gesture Recognition	Functional real-time classification of Open Palm/Fist
5	02 Jul 2026 (Thu)	Code refactoring, UI polishing, and comprehensive system testing	Bug-free, optimized codebase ready for deployment
6	03 Jul 2026 (Fri)	Final Presentation, Documentation, and Project Handover	Project completion and formal internship conclusion

3. Technical Implementation Details

3.1 Full-Stack Web App: Pneumonia Detection

The translation of the optimal MobileNetV2 model into a web application required a tightly coupled frontend and backend architecture.

Backend API (Flask): I initialized a Python Flask application (app.py). The core of the backend is the /predict route, configured to accept POST requests.

8. **Model Loading:** The pre-trained .h5 model weights (derived from the 1000-image dataset) were loaded into global memory upon server startup using `keras.models.load_model()` to minimize inference latency.
9. **Image Preprocessing:** When a request is received, the raw image byte stream is intercepted. I utilized the Python Imaging Library (PIL) to open the image, resize it to the exact dimensions expected by the model (e.g., 224 x 224), and convert it to an RGB array. The pixel values were then normalized to the [0, 1] range, mirroring the exact preprocessing pipeline used during training.

10. Inference and Response: The preprocessed tensor is passed to `model.predict()`. The resulting probability array is evaluated. If the probability exceeds 0.5, the diagnosis is classified as "Pneumonia", otherwise "Normal". The backend then serializes this string classification and the raw confidence score into a JSON payload and transmits it back to the client.

Frontend Interface (HTML/CSS/JS): The frontend was designed for extreme simplicity, acknowledging that the end-user might be a medical technician requiring rapid results.

11. HTML/CSS: I built a clean, centered interface utilizing Flexbox for responsive layout. The design features a prominent file upload zone (styled with a dashed border and hover effects to indicate interactivity) and a dedicated results display area.
12. JavaScript Logic: I implemented an event listener on the upload form. Upon submission, the default page reload is prevented via `e.preventDefault()`. A `FormData` object is constructed and populated with the selected image file. I utilized the modern `fetch()` API to asynchronously transmit this payload to the Flask `/predict` endpoint. While awaiting the response, a loading state is displayed. Upon receiving the JSON response, the DOM is dynamically updated to display the prediction (color-coded red for Pneumonia, green for Normal) and the confidence percentage.

3.2 Gesture Classification Heuristics

Building upon the real-time landmark extraction pipeline developed in Week 5, the objective was to implement semantic gesture classification. This required translating the spatial arrays into logical decisions.

I focused on a heuristic approach based on Euclidean distance calculations between specific MediaPipe nodes.

13. Fingertip to Palm Distance: To classify an "Open Palm" versus a "Closed Fist", I calculated the distance between the wrist (Landmark 0) and the tips of the index, middle, ring, and pinky fingers (Landmarks 8, 12, 16, 20 respectively).
14. Mathematical Logic: If all four calculated distances fell below a dynamically scaled threshold (scaled based on the overall bounding box size of the hand to account for distance from the camera), the state was classified as a "Fist". Conversely, if the distances exceeded a larger threshold, indicating full extension, the state was classified as an "Open Palm".
15. Real-Time Execution: This mathematical logic was injected directly into the while True video capture loop, executing on every frame immediately after the MediaPipe extraction. The resulting classification string was then rendered onto the video feed using `cv2.putText()`, providing immediate, live feedback of the system's heuristic reasoning.

4. Day-wise Work Summary

Day 31 – Monday, 28 June 2026

Backend Architecture and Flask API Development

The final week commenced with the crucial task of building the backend infrastructure. The objective was to liberate the optimal MobileNetV2 model from the Jupyter Notebook environment.

I initialized a new Python virtual environment and installed Flask, TensorFlow, and Pillow (PIL). I authored `app.py`, defining the core `/predict` POST endpoint. The primary challenge of the day was perfectly replicating the model's expected preprocessing pipeline within the Flask route. I wrote utility functions to ingest the raw file stream, convert it to an image object, resize it to 224 x 224 pixels, normalize the arrays to `[0, 1]`, and expand the dimensions to create a valid batch tensor `(1, 224, 224, 3)`.

I rigorously tested the API using Postman, sending raw image files and verifying that the server responded with accurately formatted JSON payloads containing the diagnostic prediction and confidence scores. By the end of the day, the brain of the web application was fully operational.

Day 32 – Tuesday, 30 June 2026

Frontend UI/UX Design

With a functioning backend API, Tuesday was dedicated to developing the human-computer interface. The goal was to build a professional, clean, and intuitive medical web application.

I authored the `index.html` file, structuring the DOM into logical sections: a header, an upload form, an image preview area, and a results display panel. I then focused heavily on CSS styling. I utilized CSS Flexbox to ensure the application was responsive across different screen sizes. I designed a drag-and-drop style file upload zone, implementing hover states and subtle transitions to improve the user experience. The design aesthetic was intentionally minimalist, using stark contrasts (white backgrounds, dark text, stark red/green for results) to ensure absolute clarity for the user.

Day 33 – Wednesday, 01 July 2026

Full-Stack Integration via JavaScript

Wednesday marked the convergence of Monday's backend and Tuesday's frontend. The objective was to bring the application to life using asynchronous JavaScript.

I wrote the client-side logic to intercept the form submission. I utilized the `FormData` API to package the image file and employed the `fetch()` API to make the asynchronous POST request to the Flask server. I implemented crucial UI state management—displaying a "Processing..." indicator while the network request was in flight, and dynamically injecting the returned JSON data into the DOM upon success.

A significant portion of the day was spent on error handling. I wrote logic to catch network failures, server 500 errors, and invalid file type uploads, ensuring the application degraded gracefully and provided clear error messages to the user rather than failing silently. By the end of Wednesday, the end-to-end Pneumonia Detection Web Application was completely functional.

Day 34 – Thursday, 02 July 2026

Advanced Gesture Recognition Heuristics

Having secured the web application, Thursday involved a rapid pivot back to the Computer Vision domain. The goal was to elevate the Hand Tracking module from a visualization tool to an analytical engine.

I opened the foundational OpenCV/MediaPipe script developed in Week 5. I imported the math library and wrote utility functions to calculate the Euclidean distance between two 3D points. I implemented the heuristic logic to compare the distances of the four fingertips relative to the wrist node.

The primary challenge was threshold tuning. Because the x, y, z coordinates returned by MediaPipe are normalized, I had to establish relative thresholds that remained robust regardless of how close the hand was to the camera. After significant trial and error and live testing, I established robust thresholds capable of flawlessly differentiating between an "Open Palm" and a "Closed Fist" in real-time. I used `cv2.putText` to render the classification directly onto the live video feed.

Day 35 – Friday, 03 July 2026

Refactoring, Polish, and System Testing

Friday was dedicated to the critical Software Engineering practices of refactoring and quality assurance. The objective was to prepare both systems for professional presentation and deployment.

For the web application, I conducted an extensive code review. I removed all redundant `debug print()` statements from the Flask backend. I optimized the CSS, removing unused classes and consolidating styles. I tested the application with dozens of diverse X-ray images, verifying that the frontend gracefully handled both highly confident predictions and edge cases.

For the gesture module, I refactored the Euclidean distance calculations into a clean, reusable Class structure, ensuring the code was modular and easily extensible for future gesture additions. The local directory structures were finalized, ensuring a clean separation between the frontend/, backend/, and models/ directories.

Day 36 – Saturday, 04 July 2026

Final Presentation and Project Handover

The final day of the internship was focused entirely on delivery.

I finalized the comprehensive documentation, ensuring the README.md files contained precise instructions for initializing the Python environments, launching the Flask server, and executing the real-time computer vision scripts.

During the final review meeting, I delivered a live demonstration of both projects. I successfully showcased the end-to-end flow of uploading an X-ray image via the web UI and receiving a near-instantaneous diagnostic prediction from the MobileNetV2 backend. I then demonstrated the real-time Gesture Recognition script, successfully classifying varying hand states with zero noticeable latency.

The mentorship team formally accepted the projects, marking the successful conclusion of the 6-week intensive Summer Internship program. The final codebase was pushed to the master branch of the GitHub repository, signifying the official handover of the technology.

5. Methodology

The methodology for Week 6 was rooted in Agile Software Engineering and Full-Stack development principles.

1. API-First Design: The development of the web application followed an API-first approach. By designing and rigorously testing the Flask backend independently using tools like Postman before writing any frontend code, I ensured a clear separation of concerns. This methodology guarantees that the backend is robust and decoupled, theoretically allowing it to serve multiple different frontends (e.g., a mobile app) in the future.

2. Asynchronous Client-Server Architecture: The web app utilized an asynchronous architecture via the JavaScript fetch API. This prevents page reloads during the computationally expensive image inference process, providing a seamless, modern Single Page Application (SPA) feel and significantly enhancing the user experience.

3. Heuristic Algorithmic Design: The methodology for gesture classification relied on mathematical heuristics (Euclidean distances and geometric rules) rather than training a secondary machine learning classifier on top of the landmark data. This approach was chosen to ensure absolute zero-latency execution, as complex mathematical evaluations are computationally cheaper than forward passes through neural networks.

4. Continuous Refactoring: A core methodological principle applied this week was continuous refactoring. Code was written to function first, and then immediately refactored to be clean, modular, and optimized for production deployment, adhering to professional coding standards.

6. Tools and Technologies Used

Category	Tool / Technology	Purpose
Backend Framework	Python Flask	Architecting the REST API, handling HTTP routing, and managing the server-side machine learning inference.
Frontend Layout	HTML5 & CSS3	Designing the user interface, utilizing Flexbox for responsive layouts and CSS transitions for interactivity.
Client-side Logic	JavaScript (ES6)	Handling DOM manipulation, form event interception, and executing asynchronous network requests via the fetch API.
Image Manipulation	Pillow (PIL)	Server-side image preprocessing (resizing, array conversion) to prepare uploaded files for the TensorFlow model.
Computer Vision	OpenCV & MediaPipe	Core engines driving the real-time video capture and 3D landmark extraction for the gesture module.
API Testing	Postman	Utilized for independent, rigorous testing of the Flask /predict endpoint prior to frontend integration.
Version Control	Git / GitHub	Final repository management and official project handover.

7. Results and Observations

The final week yielded the ultimate, tangible results of the internship, transforming research into fully functional software.

1. Functional Medical Web Application: The Pneumonia Detection System was successfully deployed as a robust web application. The full-stack integration was seamless, providing an intuitive interface capable of executing complex MobileNetV2 inferences and returning diagnostic results in under a second. The abstraction of the machine learning complexity behind a clean UI was a resounding success.

2. Real-Time Heuristic Gesture Classification: The Hand & Gesture Recognition module achieved its primary objective. The implementation of Euclidean distance heuristics successfully enabled the real-time classification of complex kinematic states (Open Palm vs. Fist) with zero noticeable processing latency, proving the viability of mathematical logic layered over MediaPipe extraction.

3. Production-Ready Codebase: The rigorous refactoring and testing phases resulted in a highly polished, modular, and well-documented codebase. The separation of concerns between the

frontend client and backend API ensures the software is maintainable and scalable by the Akshar AI engineering team.

8. Challenges Faced

16. **Tensor Shape Mismatches in Production:** The most significant challenge occurred during backend integration. The image preprocessing pipeline in the Flask app initially failed to match the exact tensor shape (1, 224, 224, 3) required by the .h5 model. Debugging this required deep inspection of the NumPy array dimensions at every step of the PIL transformation process to ensure the batch dimension was correctly expanded.
17. **Asynchronous UI State Management:** Managing the frontend UI states during the asynchronous fetch request was complex. Ensuring the "loading" indicator appeared immediately upon submission, remained active during network latency, and disappeared precisely when the JSON response was parsed (or an error was caught) required precise JavaScript event handling.
18. **Dynamic Thresholding in CV:** Establishing the threshold values for the gesture heuristics was challenging because the apparent distance between fingers changes based on how close the hand is to the webcam lens. I had to experiment with normalizing the calculated Euclidean distances against the overall size of the hand bounding box to ensure the logic remained robust at varying depths.

9. Learning Outcomes

The intense focus on deployment and full-stack integration during Week 6 provided crucial, industry-relevant skills:

RESTful API Architecture: Mastered the design and implementation of backend APIs using Flask. Learned how to handle multi-part form data, execute server-side logic, and serialize responses via JSON.

Full-Stack Integration: Gained practical expertise in connecting disparate software layers. Learned how to utilize asynchronous JavaScript (fetch, Promises) to bridge client-side interfaces with backend Python servers.

Model Deployment Fundamentals: Bridged the critical gap between Data Science and Software Engineering. Learned how to take a static machine learning model file (.h5) and deploy it into a dynamic, production-like environment capable of serving concurrent predictions.

Algorithmic Heuristics: Developed the ability to translate spatial data into mathematical logic, writing geometric heuristic algorithms to classify complex computer vision states without relying on secondary neural networks.

End-to-End Product Delivery: Experienced the complete software development lifecycle, from initial theoretical research and architectural planning to final UI styling, bug squashing, and formal project handover.

10. Conclusion

Week 6 marked the triumphant conclusion of the 6-week Summer Internship program at Akshar AI Ltd. The intense effort dedicated to full-stack development and algorithmic design yielded two fully functional, highly impressive software solutions.

The Pneumonia Detection System evolved from a collection of isolated Jupyter Notebooks into a seamless, robust web application. The integration of the highly optimized MobileNetV2 model behind a Flask API and a clean, responsive frontend interface demonstrates the successful translation of advanced deep learning research into a tangible, user-friendly product.

Furthermore, the Hand & Gesture Recognition module achieved semantic understanding. By layering complex Euclidean heuristics over the MediaPipe extraction pipeline, the system can now classify kinematic states in real-time. This functional module serves as a highly robust foundation for the broader Human Behavior Tracking System.

Over the course of six weeks, the progression from basic exploratory data analysis to massive-scale model scaling, and finally to full-stack web deployment, has been immense. The internship provided an unparalleled opportunity to bridge the gap between theoretical machine learning and practical, production-grade software engineering. The final projects stand as a testament to rigorous methodology, continuous learning, and aggressive, successful execution.